

SOP 60 — Venues (Perto de você)

Camada A.N.I.: Instruments-Tools (ingest_venues.py) · Versão: 1.0 (Protocol 0, 2026-09-09)
Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Manter o cadastro de bares, lojas, embaixadas, caravanas e telões (a seção `perto_de_voce` do payload) a partir da **aba venues** — que é escrita por humano no dia 1 e depois pelos próprios estabelecimentos via formulário. A tool **lê o sheet, não scrapa Google** (geocoding é fase 2). Mapa começa com as **5 cidades piloto** e planilha vazia — nada de inventar 200 bares.

Quando roda

- Venue weekly: segunda 10:00 — `ingest_venues` importa cadastros novos como `pending` (entram para revisão humana).
- Na montagem do payload (SOP 30): leitura dos `verified` da cidade default.
- Revisão humana: a qualquer momento (muda `status` no sheet).

Input

- Aba `venues` (Google Sheets/Supabase) com o contrato `RawInput.Venue`: `venue_id`, `name`, `type` (`bar|loja|embaixada|caravana|telão`), `city`, `state`, `address`, `geo.lat/lng`, `has_screen`, `status` (`pending|verified|rejected|closed`), `matchday_note`.
- Cidades piloto (constantes em `gemini.md`): Rio de Janeiro, Belo Horizonte, Recife, Salvador, São Paulo.
- Formulário de cadastro documentado (abaixo) → novas linhas na aba `venues`.

Passos determinísticos

1. Ler a aba `venues` inteira (com cache local em `.tmp/venues/` para diff).
2. Normalizar: `city` / `state` padronizados (sem acento em `venue_id`); `type` e `status` dentro dos enums da CONSTITUTION — linha com enum inválido → `rejected` com nota de motivo (nunca corrigir silenciosamente).
3. Linhas **novas** (criadas desde a última leitura) → marcar `status=pending` e avisar o editor no canal (entram na revisão humana).
4. Campos obrigatórios ausentes (`name`, `type`, `city`) → `pending` com nota; não entra no payload.
5. **Filtro do payload (SOP 30)**: só entram venues com `status=verified` E `city` igual à cidade do usuário/default; fora das 5 cidades piloto **não** entra no Daily Payload do MVP (pode existir no sheet).
6. Atualizar o cache `.tmp/venues/` e reportar: `{ "total": n, "pending": n, "verified_by_city": {...} }`.

Edge cases

- Venue mudou de cidade/nome depois de verificado → volta para `pending` (reverificação humana) na próxima leitura que detectar diff relevante.
- Dois cadastros do mesmo lugar (duplicidade por nome+endereço) → manter o mais antigo `verified`, novo vira `pending` com nota “possível duplicado”.
- Venue `closed` → nunca entra no payload; se estava em payloads antigos, some naturalmente na próxima montagem.
- Sheet inacessível → retry 3x; persistindo, payload sai com `perto_de_voce.items=[]` + `status: "empty"` e falha registrada em `progress.md`.
- Cadastro fora das cidades piloto → aceito no sheet (dado existe), mas documentado que não entra no MVP (regra do blueprint).

Formulário de cadastro (MVP — 1 formulário documentado)

- Google Form (ou equivalente) ligado à aba `venues` com campos: nome do estabelecimento, tipo (bar/loja/embaixada/caravana/telão), cidade (lista das 5 piloto + “outra”), endereço, tem telão? (sim/não), observação de dia de jogo, contato de quem cadastra.
- Todo cadastro entra como `pending` e passa por revisão humana antes de `verified`.
- Divulgação do formulário: decisão humana (não automatizar no MVP).

Rate limits / limites conhecidos

- Leituras da planilha: 1x por ciclo agendado (semanal) + leitura leve na montagem do payload; respeitar quota da API do Sheets/Supabase (fixar na Phase L).
- Nenhum scraping externo neste SOP (proibido pelo blueprint).

Testes

- Fixture manual: linha `verified` de cada uma das 5 cidades + 1 `pending` + 1 `closed` → payload (SOP 30) só recebe `verified` da cidade default; total correto no relatório.
- Phase L: validar leitura real da aba com `tools/ping_sheets.py`.